



UNIVERSITÉ
CAEN
NORMANDIE

Rapport de Projet

Jeu de cartes - Blackjack

UE: Méthodes de conception

Date de soumission : December 5, 2025

Année Académique: 2025 - 2026

Encadrement Pédagogique

Professeur (CM):	MATHET Yann
Chargé de TP:	Christophe Charrier
Chargé de TP:	Guillaume Letellier

Réalisé par

Étudiant	Numéro d'Étudiant
BENSADI Aris	22410069
CHAIB SETTI Mohamed	22310564
REZGUI Lena	22309694
LAMAIRI Mohamed Yassine	22312242

Table des matières

1	Introduction	3
2	Architecture Globale	3
3	Design Patterns	3
3.1	Command Pattern	3
3.2	Decorator Pattern	5
3.3	Strategy Pattern	6
3.4	Adapter Pattern	7
3.5	Factory Pattern	8
3.6	Observer Pattern	8
4	Conclusion	9

1 Introduction

Le projet consiste en une implémentation du célèbre jeu de cartes "Blackjack" en Java. L'objectif principal étant de concevoir une application modulaire, extensible et respectant les principes de la programmation orientée objet. Il s'agit d'une version monoposte où un joueur humain se mesure à des « robots ». L'interface graphique est réalisée avec la bibliothèque Swing.

2 Architecture Globale

L'application repose sur le motif architectural **MVC** (Modèle-Vue-Contrôleur), garantissant une séparation claire entre les données, l'interface utilisateur et la logique de contrôle.

- **Modèle** ('`blackjack.modele`', '`cartes.modele`') : Contient la logique métier (cartes, paquets, calcul des points, état des joueurs, gestion de la logique des coups ...). Il est indépendant de l'interface graphique.
- **Vue** ('`blackjack.vue`', '`cartes.vue`') : Gère l'affichage graphique (fenêtres, panneaux, rendu des cartes). Elle observe le modèle pour se mettre à jour.
- **Contrôleur** ('`blackjack.controleur`') : L'orchestrateur qui gère le flux du jeu, synchronise les tours et traite les entrées utilisateurs.

3 Design Patterns

Afin d'assurer la solidité et la maintenabilité du projet, plusieurs patterns ont été appliqués judicieusement

3.1 Command Pattern

Les actions de jeu (Tirer, Rester, Doubler, Splitter) sont encapsulées dans des objets implémentant l'interface `Coup`.

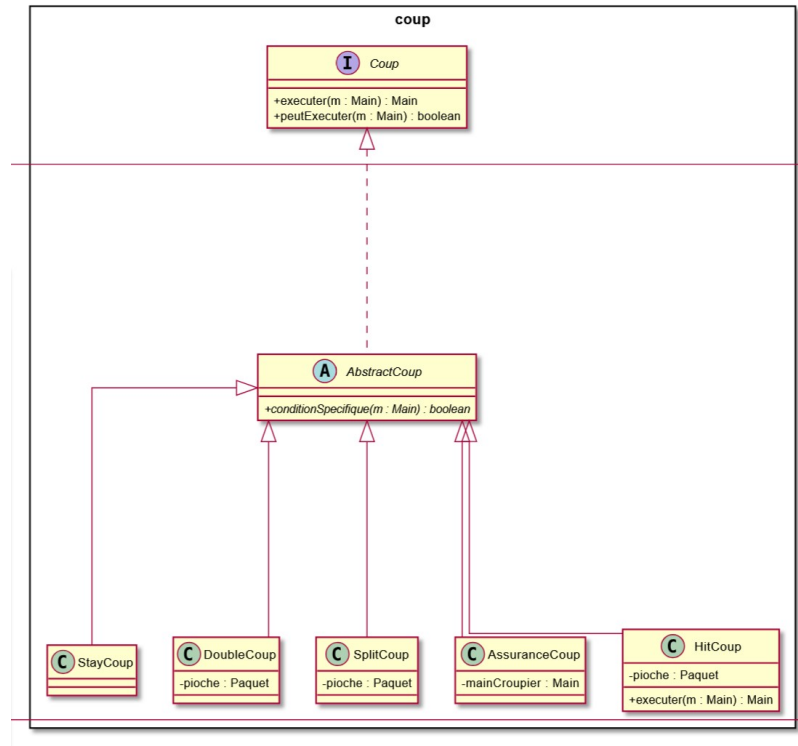


Figure 1: Diagramme de classes illustrant l'implémentation de command pattern

Mise en œuvre : Chaque coup (ex: `HitCoup`, `StayCoup`) possède une méthode `executer(Main main)` qui renvoie une nouvelle `Main`.

Intérêt : Chaque action est autonome, avec ses propres règles d'exécution, ce qui rend la gestion des coups plus modulaire. De plus, l'encapsulation de ces actions nous permet d'appliquer le ****Principe Open/Closed**** :

- L'orchestrateur du jeu (**Contrôleur**) est initialisé en lui passant la liste exacte des Coups autorisés pour cette partie.
- Cela rend le **Contrôleur** ****fermé à la modification**** (son code n'a pas besoin de changer pour gérer un nouveau coup).
- Il est ****ouvert à l'extension**** : pour ajouter un nouveau type d'action (ex: "Abandonner"), il suffit de créer la nouvelle classe `Coup` et de l'ajouter à la liste d'initialisation, sans modifier le cœur de la logique du contrôleur.

3.2 Decorator Pattern

Le Decorator Pattern joue un rôle central dans la gestion des mains des joueurs. .

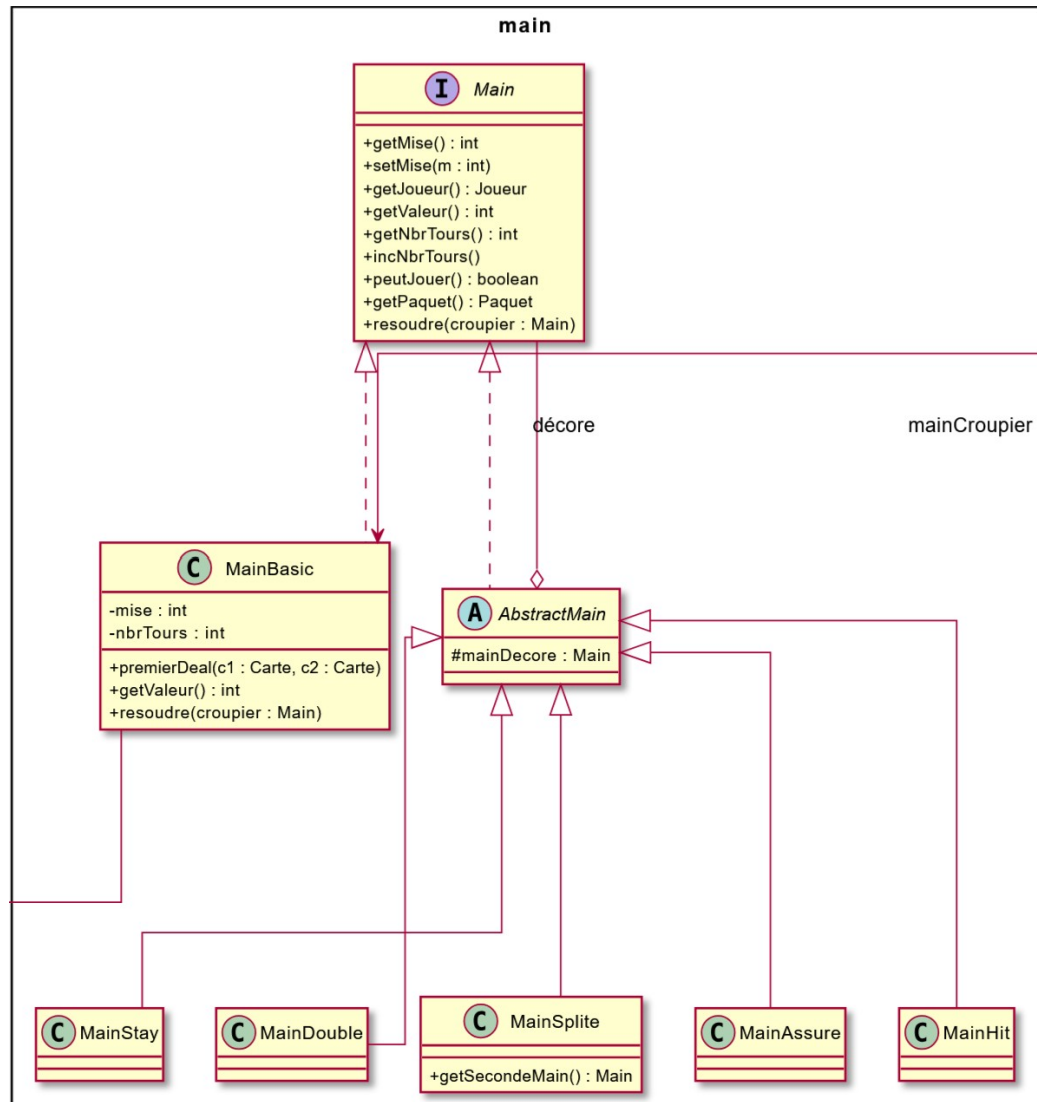


Figure 2: Diagramme de classes illustrant les pattern Decorator sur La main .

Mise en œuvre : L'interface Main définit les fonctionnalités de base communes à toutes les mains, comme accéder à la mise , récupérer le joueur détenteur de cette

dernière... La classe ‘MainBasic’ représente une main initiale, Le pattern décorateur permet ensuite de dynamiser le comportement d’une main en fonction des actions spécifiques du jeu, comme le doublement, le split ou l’assurance, sans modifier la classe de base. Ainsi, chaque action du joueur permet de « décorer » la main avec le comportement adéquat tout en conservant l’interface commune.

- ‘MainHit’ : Ajoute une carte.
- ‘MainDouble’ : Double la mise et ajoute une dernière carte.
- ‘MainAssure’ : Gère l’état d’assurance contre le Blackjack du croupier.
- ‘MainSplitte’ : Gère la division d’une main en deux.

Intérêt : Cela permet de conserver un historique implicite des actions et d’ajouter dynamiquement des comportements ou des restrictions (par exemple, impossible de tirer après un "Double") sans complexifier une classe unique avec de multiples drapeaux booléens.

3.3 Strategy Pattern

Pour la gestion des joueurs non humains (‘JoueurIA’), nous avons utilisé le :Pattern Strategy.

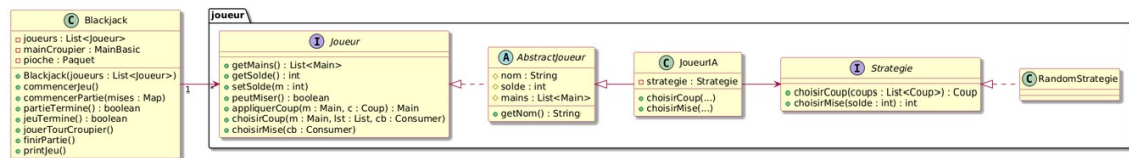


Figure 3: Diagramme de classes illustrant l’implémentation du strategy pattern

Mise en œuvre : L’interface ‘Strategie’ définit les méthodes ‘choisirCoup’ et ‘choisirMise’. La ‘RandomStrategie’ implémente une de ses logiques. **Intérêt :** Cela permet d’interchanger le niveau d’intelligence des bots (Aléatoire, Prudent, Compteur de cartes) sans modifier la classe ‘JoueurIA’ elle-même rendant le code plus flexible et plus ergonomique à maintenir..

3.4 Adapter Pattern

Afin d'assurer l'affichage des joueurs et de leurs soldes dans l'interface graphique, une classe `ModeleSolde` a été implémentée. Elle sert d'adaptateur entre le composant Swing standard `JTable` et notre classe métier `Joueur`.

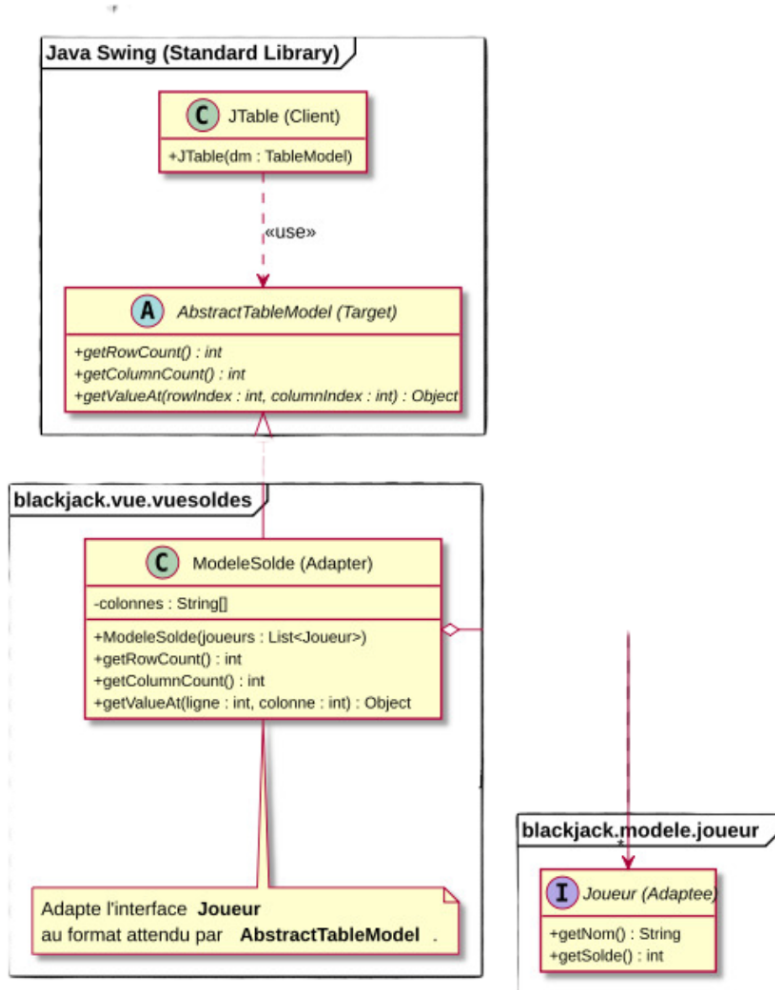


Figure 4: Diagramme de classes illustrant l'implémentation du Adapter Pattern

Mise en œuvre : La classe `ModeleSolde` agit comme un pont. Elle hérite de `AbstractTableModel` pour satisfaire les attentes de la `JTable` et encapsule une liste d'objets `Joueur`.

Intérêt : L'intérêt de ce pattern est qu'il adapte l'interface de la classe `Joueur` à celle attendue par la `JTable`.

- Grâce à cette adaptation, la `JTable` peut afficher des données des objets `Joueur` sans qu'il soit nécessaire de modifier la classe `Joueur` elle-même.

3.5 Factory Pattern

Afin de simplifier l'instanciation des jeux de cartes, la création des objets `Paquet` a été déléguée à une classe usine dédiée : `PaquetFactory`.

Mise en œuvre : La classe `PaquetFactory` centralise la logique de création. Elle expose des méthodes spécifiques comme `creerPaquet52()` ou `creerPaquet32()`. Ces méthodesinstancient en interne toutes les `Cartes` nécessaires (boucles sur les couleurs et hauteurs), les assemblent dans un objet `Paquet`, et retournent ce dernier prêt à l'emploi.

Intérêt : Ce pattern offre une ****encapsulation de la logique de création****.

- Le code client n'a pas besoin de connaître la complexité de la génération d'un jeu de 52 cartes.
- Il assure la cohérence des objets créés (un paquet sort de l'usine toujours complet et valide).
- Il facilite l'ajout futur de nouveaux types de paquets (ex: jeu de Tarot) sans modifier le reste de l'application.

3.6 Observer Pattern

Pour relier le modèle à la vue tout en maintenant un faible couplage, nous avons implémenté un système d'écouteurs basé sur le pattern Observer

Mise en œuvre : Les classes du modèle (comme `'Paquet'`) étendent `'AbstractEcoutable'`. Les vues (comme `'VuePaquet'`) implémentent `'Ecouteur'`. Lorsqu'une carte est ajoutée ou retirée, la méthode `'notifierChangement()'` déclenche le rafraîchissement automatique de l'interface.

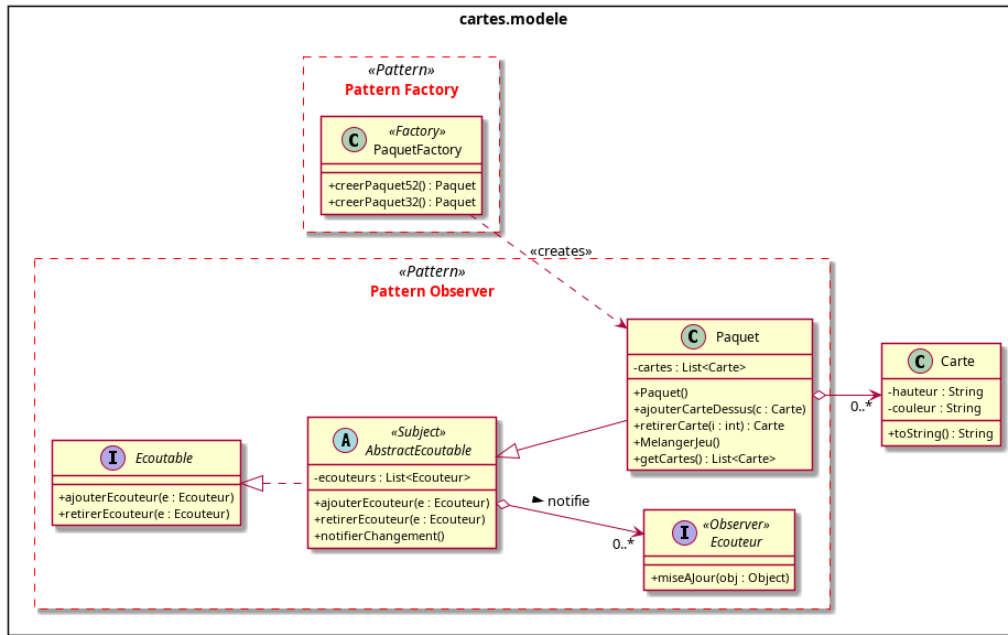


Figure 5: Diagramme de classes illustrant l'implémentation des pattern factory et observer

4 Conclusion

Ce projet a permis de mettre en pratique des concepts avancés de conception orientée objet. L'architecture découplée en MVC rend l'application plus claire et plus robuste. De plus, l'intégration de plusieurs design patterns (Decorator, Strategy, Adapter, Command, Factory, Observer) a permis d'obtenir un code plus modulaire, réutilisable et extensible.